CSCI U550 AI and Cybersecurity | Fall 2026 | Dr. Qingli Zeng

[Project 1 Report: What We Built and What We Found]

Project 1 Report Template | Target length when filled in: 6–8 pages as PDF

Team members	Track (A or B)	Date
Ana Madrigal, Sydnie Dailey, Katherine Vykhovanets	Track A: Phishing URLs	10/19/2026

1. Introduction

Phishing is a common cybersecurity issue where attackers trick users into visiting fake or malicious websites, often by using URLs that appear trustworthy or imitate real services. These links can be used to steal login information or other sensitive data. A phishing URL detector can help identify suspicious links before someone visits them, which can be useful for both individuals and organizations. In this project, we built a machine-learning detector that uses information from the URL itself to classify URLs as either phishing or legitimate.

One challenge with phishing detection is that phishing URLs and techniques change over time. Attackers can change the domains, hosting services, and URL patterns they use, meaning a model trained on older examples may not recognize newer ones. Therefore, timestamps are important because they allow us to test whether a detector can detect phishing URLs that appear later. A random split may contain many similar patterns in both training and testing, making performance look better than it would in future data.

Because of this, evaluation design can matter as much as the choice of model. We compared random stratified, temporal, and domain-grouped splits. The temporal split trains on earlier samples and tests on later ones, while the domain-grouped split prevents the same registered domains from appearing in both training and testing. These evaluations help show if the detector learned patterns that generalize instead of patterns that only work on familiar data.

Our results showed a major difference between the random and temporal evaluations. The best model achieved an F1 score of 0.905 and ROC-AUC of 0.983 on the random split, compared with 0.574 and 0.500 on the temporal split. Recall at 1% FPR dropped from 89.70% to 0.96%, while recall at 0.1% FPR dropped from 64.07% to 0.21%. This shows that strong performance on randomly selected test data did not translate to newer phishing URLs, making changes in phishing over time one of the most important findings of the project

2. Data

Our final dataset was built from phishing URLs collected using the provided collect_phish.py script and legitimate URLs from the provided benign_urls.csv. The phishing collection included the history backfill and our live pulls recorded in collection_log.csv, while the legitimate URLs came from the older pre-collected corpus provided with the project starter kit. We started with 51,209 phishing URLs and 344,800 legitimate URLs in separate CSV files. After cleaning and removing 145 total near-duplicates, 51,072 phishing URLs and 344,792 legitimate URLs remained.

Stage	Phishing	Legitimate
Raw Dataset	51,209	344,800
After Deduplication	51,072	344,792
Near-duplicates removed	137	8
Final after subsampling	51,072	51,072

Final class share	50%	50%

Before combining the two classes, we cleaned them separately while keeping the original URLs unchanged. Our preprocessing handled differences such as trailing slashes, query-parameter order, tracking parameters, hostname capitalization, percent-encoding, and URL fragments. URLs that became identical after these normalization steps were treated as near-duplicates, which removed 137 phishing URLs and 8 legitimate URLs. The original and processed datasets were kept separately so we could still check how each URL changed during preprocessing.

Two of the most important curation decisions involved reducing shortcuts that came from how the source files were built. All of the original phishing URLs contained a scheme (http:// or https://), while none of the legitimate URLs did, so the scheme was removed from the phishing URLs used for modeling. The original classes also had very different path structures: 45.35% of phishing URLs had a path compared with 86.09% of legitimate URLs. We therefore kept all 51,072 phishing URLs and subsampled the legitimate class by path-depth bucket, giving both classes the same size and path-depth distribution.

The final dataset contains 102,144 URLs, split evenly between 51,072 phishing and 51,072 legitimate samples. Some differences were intentionally left in the data instead of being forced to match. For example, the top 20 domains account for 49.99% of phishing URLs compared with 17.97% of legitimate URLs, and the two classes still have different TLD distributions. The legitimate corpus is also older than the phishing collection, so these differences remain possible limitations or shortcuts during modeling. Additional collection, preprocessing, and comparability details are provided in the datasheet.

3. EDA Findings

Our three most informative findings are: Figure 1: URL Length Differences, Figure 2: IP Address Usage, and Figure 6: Average Number of Dots. A weakness over time can come from phishing URLs changing over time as attackers develop new ways to avoid detection. This is important for security analysts because phishing detection methods need to keep up with new attacks. Regularly updating the dataset with newer phishing URLs can help improve detection accuracy and prevent the model from relying only on older phishing patterns.

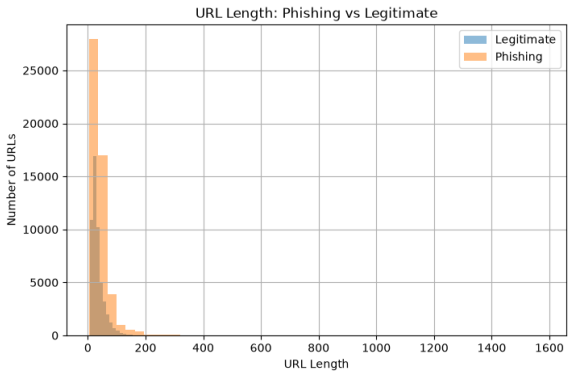


Figure 1: URL Length

One of the important features examined in this project was the URL length. The graph compares the distribution of legitimate and phishing URL lengths to help identify differences between the two groups. Phishing websites may use longer URLs to hide suspicious information, include misleading website names, or add unnecessary characters. This matters to security analysts because unusually long URLs can be an indicator of suspicious activity. However, legitimate websites can also have long URLs, especially when they contain tracking information or multiple directories. URL length should therefore be used alongside other features rather than as the only way to identify phishing URLs.

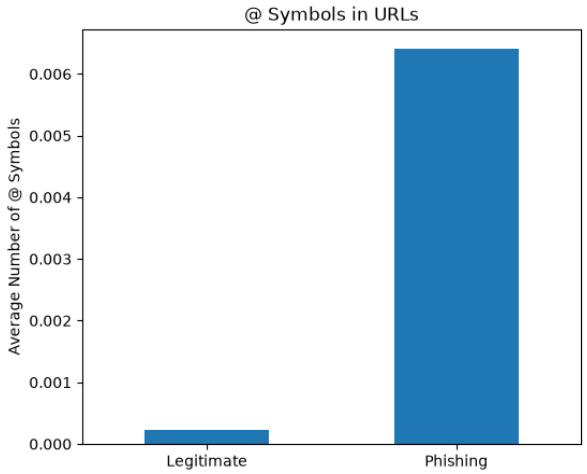


Figure 2: IP Address Usage

Another important feature was whether URLs used numerical IP addresses instead of traditional domain names. The graph compares the percentage of legitimate and phishing URLs that use IP addresses. This is useful because attackers sometimes direct users to IP addresses rather than recognizable website domains. For security analysts, identifying URLs that use IP addresses can help flag potentially suspicious websites for further investigation. However, not every URL containing an IP address is malicious, and many phishing websites use normal domain names. This means IP address usage can be helpful as an additional detection feature but should not be treated as definite proof of phishing URLs.

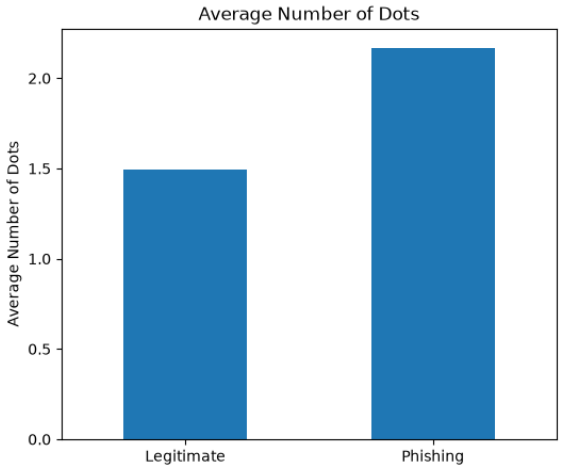


Figure 3: Average Number of Dots

The third finding focuses on the legitimate dataset and how well it represents real-world internet activity. The graph compares the average number of dots in legitimate and phishing URLs, which provides a basic measurement of URL structure. One limitation of the legitimate dataset is that it may contain simpler website addresses than those users normally encounter. Real users often visit websites with multiple subdomains, file extensions, and complicated URL paths. If the legitimate dataset does not include enough of these examples, a phishing detection model could incorrectly classify complex but safe URLs as malicious. This is important because security analysts need datasets that represent realistic browsing behavior to reduce false positives and improve detection accuracy.

4. Method

Lexical Feature Set:

1. We chose URL length because we noticed that many of the legitimate URLs have a closer range of lengths compared to the phishing URLs that have a larger and more diverse range.
2. The second feature is hostname length because many of the phishing URLs have longer domains than can be easily and quickly changed.
3. The frequency of ".", "\", "-", and "@" in the hostname was chosen because we noticed that many of the phishing URLS have a larger amount than the legitimate URLS.
4. We looked at the number of query parameters in each URL because many of the legitimate URLs don't have query parameters, unlike some of the phishing URLs.
5. The sixth feature we chose is the TLD type because most phishing URLs have a short lifespan and need a TLD that can be easily changed. We made a list based on the most popular TLDs in phishing URLs right now and the most popular TLDs in the phishing URLs in our data.
6. We also decided to look for suspicious keywords using a CSV containing 75 common keywords in phishing URLs.
7. We chose the ratio of encoded characters because we noticed that the phishing URLs that are long, tend to have many encoded characters.
8. Another feature we chose in the entropy of the URL. Most phishing URLs have a higher entropy than the legitimate URLS
9. We chose the number of subdirectories because we noticed that many of the phishing URLs contain either a lot of subdirectories or they don't have a path at all, while the legitimate URLs are all in a similar range for the number of subdirectories they have.
10. The final lexical feature that we chose was the number of numbers in the URL. This is because we observed that many of the phishing URLs will use strings of numbers to create many near-identical URLs.

Character n-gram

The n-gram will analyze characters that have an n-gram greater than or equal to 3 and less than or equal to 4, convert all characters to lowercase, and only analyze 200,000 features max. Since the dataset is so large, it took a few tries to get the best n-gram for the dataset. To make this process faster, I lessened the n-gram range from (3,5) to (3,4), and I decreased the number of features from 500,000 to 200,000

The Models

We used both the Logistic Regression and Random Forest models for our dataset. Logistic Regression is required, and we chose the Random Forest model because we are familiar with working with it and because it works well with the character n-gram.

Tuning

The logistic regression model was tuned using the validation slices and a for loop for the C values 0.01, 0.1, 1, and 10. The Random Forest Model was only tuned briefly by trying out different estimator amounts and max feature types. This is because in other settings, the Random Forest model would take between 5-10 minutes to train and produce results from the validation slices. Eventually, we found the best parameters that cut down time significantly, and we trained that model and used it for testing.

The 3 Data Splits:

The random split is comprised of 20% test data and 80% training data split randomly. The 80% training data was split into 20% for the validation slice and 60% remaining for tuning the models. The cutoff for the temporal split was calculated by ordering the URLs from the oldest timestamp to the most recent and then splitting them by their indexes so that the oldest 80% of URLs are training data and the newest 20% of URLs are the testing data. Lastly, the grouping key for the domain split is the combination of the second-level domain and the TLD, which will ensure that URLs with the same domain don't end up in both the testing and training data.

5. Evaluation

Split	Precision	Recall	F1	ROC-AUC	PR-AUC	Recall @ 1% FPR	Recall @ 0.1% FPR
Random stratified	0.996004	0.829646	0.905245	0.980961	0.984864	0.884570	0.6494635
Temporal	0.503546	0.653353	0.568750	0.508718	0.508386	0.010181	0.001860
Domain-grouped	0.997022	0.820204	0.900011	0.983547	0.986737	0.893788	0.607290

Why The Random Forest Model was Chosen:

The logistic regression model is linear and weaker compared to the random forest model, which also handles character-level TF-IDF better than the logistic regression model. Additionally, the random forest model is more robust, leading to better differentiation between phishing and legitimate URLs and higher metrics overall.

Why the Models with A Temporal Split Did Poorly:

The Temporal split has a precision of 0.503546, recall of 0.653353, PR-AUC of 0.508386, and a ROC-AUC of 0.508718. The temporal split yields worse results than the other splits because newer phishing methods are created often and they evolve over time. The model would have a difficult time classifying the test URLs since the training URLs are older and likely different which leads to a low ROC_AUC and PR_AUC. Additionally, this model as basically zero recall at 1% and 0.1% FPR so it is not good enough to be implemented.

Why the Domain and Random Splits Most Successful:

The random split lead to a better representation of the data than in the temporal split because that data reflected past and present phishing URL structures. This is why the random slit has a high precision of 0.996004, recall of 0.829646 and a high PR-AUC of 0.984864. The domain split led to similar results as the random split because the dataset contains a lot of URLs with the same path and very similar domains. Therefore, if these types of URLs are split amongst the testing and training data, then the model will be more precise. This is why the domain split produces a high PR-AUC of 0.986737.

Overall Results.

The most successful performance came from the Random forest model with the domain and random splits because the training data for both reflected more patterns in the testing data. The temporal split had a subpar performance because the data was gathered at different times and subsequently led to the patterns in the newer URLs being unfamiliar to the model.

Base Rate Check using the Random Forest Random Split Model:

FPR	Recall	Alarms per Day	P (Phishing | Alarm)
1%	0.884570	500392.285	0.0008838765369853774
0.1%	0.6494635	50319.73175	0.006453368066692843

These results indicate that this detector cannot realistically be deployed as is. The alarms per day of for the FPR to be 1% and 0.1% respectively are 500392.285 and 50319.73175, which are both too many false positive URLs for a team to be looking through. Both P values show that the team would spend too much time looking for only a handful of true alerts, which would waste too much time and money.

6. Error Analysis and Discussion

#	FN / FP	Sample (defanged if a URL)	One-line hypothesis
1	FN	www.roblox.com.pt/games/11378 0042580709/TRADING-Beach-Escape-1-Speed?game_id=11378004258070 9&game_name=TRADING-Beach-Escape-1-Speed&privateServerLinkCode=9 9382535900423182128231632453729	While this URL seems abnormal with some features such as its length and amount of parameters, it may score normally with the rest of the lexical features and appear legitimate.
2	FN	15881.xyz	Other than the suspicious TLD and the number of digits, this URL would score normally on lexical features, and the character n-gram could not have spotted any patterns due to the short length of the URL
3	FN	68105.xyz	Similar to the previous URL, this URL ranks normally in its lexical features, and it may have been too short for the character n-gram to classify any patterns
4	FN	vmi3479190.contaboserver.net/E	This URL has a lot of dots; however, it could seem normal due to scoring normally in most lexical features, like a non-suspicious TLD, short length, and small number of subdirectories.
5	FN	www.roblox.com.mu/communities/3347619647/vdev-script-group	The URL has a lot of parameters, dots, digits, and subdirectories however, it can score normally for other lexical features, and the character n-gram representation may not have classified patterns for certain brands
6	FP	music.tgrex.com/album/vesuvius	The length of this URL, the number of dots, and the two subdirectories are similar to some of the phishing URLs, and the character n-gram could have identified its character sequences incorrectly
7	FP	mylife.com/zip-codes/b3-452699	This URL has a lot of digits and dashes that could make it resemble a phishing URL, and it is also a similar length to the other URLs with a false positive.
8	FP	jigsaw.com/20100131_20100206_1_1/recently_added_contacts.xhtml	This URL has some abnormal features such as many digits, more than one subdirectory, and a small hostname and the digits may have caused the character n-gram to classify this URL incorrectly.
9	FP	nicolenasbandb.com	The URL is short and has no digits nor subdirectories, so it may not have triggered many lexical features. However, the n-gram may not be able to classify the patterns correctly.
10	FP	randomconnections.com?p=975	For this URL, the n-gram could have interpreted the lone parameter as a malicious pattern, and the length of the URL could be like those classified as phishing URLS

Groups and Patterns

Patterns observed in the false negatives include the URLs containing numbers and having a large variety of lengths. All the URLs also contain some numbers which should be an indicator that they are phishing however, the lexical features could have been misinterpreted, and many legitimate URLs could contain a similar number of numbers. Additionally, the lengths of the URLs vary a lot, so maybe a lot of the training phishing URLs had a smaller range of their lengths.

The patterns observed in the false positive group are their length, the low entropy, and unusual tokens, Unlike the false negative group, this group of URLs all have a similar length that could have been innocently marked as a trait of phishing URLs. Also, these URLs don't seem random, nor do they have a lot of digits so they would have been marked as having low entropy. Lastly, each URL has different tokens that may have been handled as if they were phishing by the character n-gram.

7. Limitations and Next Steps

One limitation of our dataset is that the legitimate URLs came from an older pre-collected corpus, while the phishing URLs came from a more recent live collection. Because phishing patterns change over time, the two classes do not represent the same time period. We also found differences that remained even after preprocessing. For example, the phishing class was much more concentrated among a smaller number of domains, with the top 20 domains accounting for 49.99% of phishing URLs compared with 17.97% of legitimate URLs. The TLD distributions were also different between the classes. Although we corrected clear shortcuts such as scheme presence and path-depth differences, these remaining differences could still influence what the model learns.

The temporal evaluation showed another major limitation: the model did not generalize well to newer phishing URLs. Recall at 1% FPR dropped from 89.70% on the random split to only 0.96% on the temporal split, showing that performance on randomly selected test data can greatly overestimate performance on future URLs. As a priority, a production team should collect newer legitimate and phishing data from similar time periods. Next, the model should be regularly retrained and reevaluated on newer data as phishing patterns change. Also, the collection should be expanded to include more sources and reduce dependence on a small number of highly concentrated hosting domains. Temporal and domain-grouped evaluations should continue to be used before deploying updated models.

8. Team Contributions

Member	Main contributions (notebooks, figures, report sections)	Share of work (%)
Ana Madrigal	Collection Log, EDA notebook, Report, Slides, and Peer Evaluation Form	33.33
Sydnie Dailey	Collection Log, Modeling Notebook: features, models, and three-way evaluation, Report, Slides, and Peer Evaluation Form	33.33
Katherine Vykhovanets	Collection Log, Datasheet & Curation, Dataset sample, Report, Slides, and Peer Evaluation Form	33.33
Total		100%

9. AI-Use Disclosure

Member	Tool(s)	Used for	How the output was verified
Ana Madrigal	ChatGPT	Understanding the usage of code as well as fixing code compiler errors	The output was verified by verifying the code could run in PyCharm with no errors.

Member	Tool(s)	Used for	How the output was verified
Sydnie Dailey	Copilot	Generating ideas for lexical features and understanding how to design temporal and domain splits	The output was verified by testing in VS Code and altering
Katherine Vykhovanets	ChatGPT	Used it to help generate scripts that would clean URLs and organize data, as well as debugging	Checked to see if the output was what was expected

10. References

[1] simplerhacking, (2023), Phishing-Keyword-List[Excel], Phishing-Keyword-List/Evilginx Keywords.xlsx at main · simplerhacking/Phishing-Keyword-List

[2] OpenAI. (2026). *ChatGPT* (GPT-4o version) [Large language model]. openai.com

[3] Microsoft. (2026). *Microsoft Copilot* (M365 version) [Large language model]. microsoft.com